

المحتويات

1	كيف بُني هذا المشروع - شرح خطوة بخطوة
2	0. الهدف (ماذا طلبت المهمة)
2	1. قرارات التصميم (الـ "لماذا")
3	2. خط المعالجة، مرحلة مرحلة
3	المرحلة A - الضبط والمانيفست (manifest.py، config.py)
3	المرحلة B - الحصول على الإشارات (الاستيعاب)
4	المرحلة C - التقطيع (segment_signal ingest_real.py)
4	المرحلة D - المخططات الطيفية (scalogram.py)
4	المرحلة E - تقسيم خال من التسرب (split.py)
4	المرحلة F - موازنة الفئات (balance.py)
4	المرحلة G - تحميل البيانات (data_loader.py)
5	المرحلة H - النموذج (model.py)
5	المرحلة I - التدريب (train.py)
5	المرحلة J - التقييم (evaluate.py)
5	المرحلة K - تدريب الاندماج (train_fusion.py)
5	المرحلة L - تجارب الحذف (experiments.py)
6	3. النتائج التي حصلنا عليها (بيانات KAIST الحقيقية، تسجيلات مُحْتَجَزَة)
6	4. الجودة وقابلية إعادة الإنتاج والتغليف
6	5. المخرجات المنتجة (docs/build/)
7	6. دروس البيئة (لتنمكّن من إعادة التشغيل)
7	7. الترتيب الزمني الدقيق لما بُني

كيف بُني هذا المشروع - شرح خطوة بخطوة

سردٌ كامل وأمين لـ ماذا بُني، ولماذا اتُّخذ كل قرار، وكيف تتلاءم القطع معاً. اقرأ هذا مع README.md (البداية السريع) و docs/report/report.md (التقرير الرسمي). هذه الوثيقة هي "مذكرات الهندسة".

سلسلة الأدوات في جملة واحدة: يعمل المشروع من بدايته إلى نهايته بلغة Python فقط (PyWavelets) لتحويل الموجات، و TensorFlow/Keras للشبكة العصبية الالتفافية). أما شيفرات matlab/ فهي بديل اختياري، وليست تبعية - كل نتيجة مذكورة أُنتجت بواسطة Python.

0. الهدف (ماذا طلبت المهمة)

بناء نموذج: 1. يأخذ إشارات محرك PMSM (التيار و/أو الاهتزاز)، 2. يحول نوافذ إشارة قصيرة إلى صور مخططات طيفية موجية (عبر تحويل المويجات المستمر، CWT) 3. يصنّف تلك الصور بـ شبكة عصبية التلافيفية (CNN) لكشف الأعطال وتحديدّها (سليم مقابل قصر بين اللّفات، إضافةً إلى إزالة المغنطة / الحمل الزائد كفتتين إضافيتين).
المُخرجات: مجموعة بيانات صور موسومة، شيفرة تدريب، نموذج CNN نهائي، أشكال نتائج، تقرير 25-35 صفحة، وعرض تقديمي 15-20 شريحة.

1. قرارات التصميم (الـ “لماذا”)

قبل كتابة الشيفرة، تُبنت هذه الخيارات:

لماذا	الخيار	القرار
مجاني، قابل لإعادة الإنتاج، بلا تراخيص؛ TensorFlow + PyWavelets يغطيان كل شيء..	Python (اختياري) MATLAB	اللغة
الحقيقية للمصدقية؛ الاصطناعية للتحقق الفوري وللّفات الناقصة. لمقارنة أيّ حسّاس يكشف الأعطال أفضل - سؤال بحثي حقيقي. يطابق المهمة؛ الحقيقية تغطّي اثنتين، والاصطناعية الاثنتين الأخريين.	بيانات حقيقية + اصطناعية + محاكاة MATLAB (اختياري) التيار والاهتزاز معاً	مصادر الإشارة
جدول واحد يربط كل إشارة -> مخطط طيفي -> تسمية -> تقسيم. لا حالة خفية.	4 (سليم، بين اللّفات، إزالة مغنطة، حمل زائد) data/manifest.csv	القنوات
كل معامل في ملف واحد؛ الوحدات لا تُبنت القيم في الشيفرة. اكتشاف الأخطاء مبكراً؛ إثبات الصحة لأثرٍ بحثي.	config.yaml	الفئات
	TDD (الاختبارات أولاً)	مصدر الحقيقة الوحيد
		الضبط
		المنهجية

منطق التصميم الكامل في docs/superpowers/specs/ وخطة الخطوات في docs/superpowers/plans/.

2. خط المعالجة، مرحلةً مرحلة

تدقّق البيانات: إشارة خام -> تقطيع -> مخطط طيفي CWT -> CNN -> فئة العطل. وكل مرحلة وحدة Python واحدة تحت python/

المرحلة A - الضبط والمانيفست (manifest.py, config.py)

- config.py يحمل config.yaml ويتحقّق من المفاتيح المطلوبة (يرفع خطأً إن غابت).
- manifest.py يعرف الأعمدة dataset_name, fs, severity, class, signal_type, source, signal_id, scalogram_path split, recording_id, CSV الواحد.

المرحلة B - الحصول على الإشارات (الاستيعاب)

ثلاثة مصادر قابلة للتبديل، يكتب كلّ منها صفوفاً في المانيفست:

1. مجموعة KAIST الحقيقية (ingest_mendeley.py) - المصدر الأساسي.
 - المجموعة: (Mendeley Faults Stator with PMSM 3-phase of Dataset Current & Vibration: rgn5brrgrn, CC-BY-4.0) تيار @ 100 kHz، اهتزاز @ 25.6 kHz، بصيغة TDMS؛ الحالات: عادي، بين اللّفات، بين الملفات، بعدة شدّات.
 - يحلّل المحلّ اسم كل ملف (1000W_0_00_current_interturn.tdms -> القدرة، الشدّة، الطريقة، العطل)، ويقرأ TDMS عبر npTDMS، ويخفّض المعدل إلى 10 kHz (FIR مضاد للتشويه)، ويقطّع إلى نوافذ 0.5 ثانية بترابك 50%، ويحدّد 50 مقطعاً لكل تسجيل (بخطوة متساوية تغطّي التشغيل كله)، ويكتب مقاطع npy. وصفوف المانيفست.
 - خريطة التسمية: الشدّة 0% -> Healthy؛ inter-coil/inter-turn -> InterTurn.
 2. المولّد الاصطناعي (simulate.py) - يبني إشارات تيار للفتات الأربع ببصمات MCSA مدفوعة فيزيائياً (توافقيات، نطاقات جانبية، دون-توافقيات). يمنح تحقّقاً فورياً والأمثلة الوحيدة لإزالة المغنطة/الحمل الزائد.
 3. محاكاة MATLAB (matlab/) - تصدير إشارات Simscape + FOC اختياري.
- ملاحظة عن تدقيق البيانات: قيّمت مجموعة عامة ثانية (Zenodo 13974503) ورُفضت لأنها بيانات جدولية بمعدل 10 Hz - بطيئة جداً على تحويل الموجات. هذا موثّق في docs/data-audit.md. ومعرفة لماذا رُفضت مجموعة بيانات سؤال محتمل في المناقشة.

المرحلة C - التقطيع (segment_signal ingest_real.py)

دالة نقية: تقطع مصفوفة أحادية البعد إلى نوافذ متراكبة، مُحْتَبَرَة وحدوياً بمعزل عن أيّ بيانات لإثبات صحة حساب النوافذ.

المرحلة D - المخططات الطيفية (scalogram.py)

- `compute_scalogram()` يطبق موجة Morlet المركبة (cmor1.5-1.0) على 128 قياساً (تباعد هندسي 4->256) عبر `pywt.cwt`، مُعيداً المقدار `|CWT|`.
- `save_scalogram_png()` يُسوّي، ويُسَقِّط عبر خريطة ألوان `jet`، ويحفظ PNG ملوّنة 224x224 تحت `.data/scalograms/<channel>/<class>/`.
- `generate_scalograms()` يرسم كل صف في المانيفست وهو قابل للاستئناف (يُخَطّي الصور المرسومة سلفاً). أنتجت البيانات الحقيقية 3,150 مخططاً طيفياً.

المرحلة E - تقسيم خالٍ من التسرب (split.py)

- `assign_splits()` يجمع حسب `recording_id` ويسند تسجيلات كاملة إلى تدريب/تحقق/اختبار (70/15/15)، طبقاً حسب الفئة، مستقلاً لكل قناة.
- لماذا التجميع مهم: النوافذ المتجاورة من تسجيل واحد متطابقة تقريباً. التقسيم العشوائي يضع شبه-مكررات في التدريب والاختبار معاً -> تسرب بيانات -> درجات مرتفعة زوراً. التجميع حسب التسجيل يمنع ذلك.
- إصلاح دقيق: مع 4 تسجيلات سليمة فقط، أرسل التقريب الساذج كلّها إلى التدريب. عدّلت الدالة لتضمن تسجيلاً واحداً على الأقل لكل تقسيم لأيّ فئة لديها ما يكفي من التسجيلات. مشمول بالاختبارات.

المرحلة F - موازنة الفئات (balance.py)

- البيانات الحقيقية شديدة اللاتوازن (~4 سليمة مقابل ~60 معطوبة). بالتدريب الخام، تنبأ الشبكة بـ "عطل" لكل شيء -> دقة 80% لكن كشف سليم 0% (انهيار الفئة الأكثرية).
- `balance_df()` يقلّل عيّنات الأكثرية في التدريب/التحقق فقط؛ أما الاختبار فيبقى على توزيعه الطبيعي لتعكس المقاييس الواقع.
- لهذا نبلّغ الدقة المتوازنة و `macro-F1`، لا الدقة الخام.

المرحلة G - تحميل البيانات (data_loader.py)

- `class_to_index()` (نقية، مُحْتَبَرَة) تربط أسماء الفئات -> أرقام.
- `make_dataset()` يبني `tf.data.Dataset` يقرأ PNG، ويعيد التحجيم إلى `image_size`، ويسوّي إلى `[0,1]`، ويخلط (تدريب)، ويعزّز اختياريّاً (انعكاس أفقي عشوائي). يُستورد TensorFlow كسولاً لتبقى الدوال النقية قابلة للاختبار دونه.

المرحلة H - النماذج (model.py)

- build_cnn() - الشبكة المرجعية أحادية القناة: ثلاث كُتل Conv->ReLU->MaxPool (32، 64، 128 مرشحاً) -> Softmax. -> Dense(128) -> Dropout(0.5) -> Flatten
- build_fusion_cnn() - ثنائية الفرع: فرع التفاني لكل قناة (تيار + اهتزاز)، ينتهي كلٌّ بـ **Pooling Average Global**، ثم يُدمجان، ثم رأس كثيف مشترك. (GAP) وليس Flatten، لأن تسطيح خريطتي 26x26x128 يُنشئ طبقة كثيفة ضخمة استنزفت ذاكرة الجهاز.

المرحلة I - التدريب (train.py)

- train_from_df() يبني مجموعتي التدريب/التحقق، ويحسب اختياريًا أوزان الفئات بالتردد العكسي، ويترجم بـ Adam + categorical sparse، cross-entropy، ويُدرّب مع patience=5، **EarlyStopping** (restore_best_weights).
- train() يربط config.yaml، ويطبّق الموازنة، ويحفظ نموذج keras. وسجل التدريب.

المرحلة J - التقييم (evaluate.py)

- metrics_from_predictions() (نقية، مُحْتَبَرَة) تحسب مصفوفة الالتباس وتقرير التصنيف؛ وتُمرّر labels= لتعمل حتى عندما تظهر فئتان فقط من أصل أربع في الاختبار الحقيقي (خطأٌ وُجد وأُصلح).
- main() يحلّل النموذج، ويتنبأ على تقسيم الاختبار، ويحفظ صورة مصفوفة الالتباس وتقرير JSON لكل فئة.

المرحلة K - تدريب الاندماج (train_fusion.py)

- pair_manifest() يقرن كل مخطط تيار بمخطط اهتزاز للحالة والمقطع ذاتهما، ويقسم على مستوى الحالة بحيث تقع طريقتا الحالة في التقسيم نفسه (لا تسرب).
- يدرب/يقيم النموذج ثنائي الفرع، يحفظ cnn_fusion.keras وتقريره.

المرحلة L - تجارب الحذف (experiments.py)

- يجري ثلاث دراسات على البيانات الحقيقية: (1) الموازنة، on/off (2) حجم الصورة 96/160/224، (3) منحني التعلم عند 25/50/100% من مجموعة التدريب.
- تفصيل هندسي: كل تشغيل في عملياته الفرعية الخاصة (ليحرّر TensorFlow الذاكرة بين التشغيلات - فتشغيل 16 تدريباً في عملية واحدة استنزف الذاكرة)، والنتائج محفوظة بنقاط تفتيش بعد كل تشغيل (فلا يضيع شيء عند تعطل أو سبات؛ يستأنف من نقطة التفتيش).
- يكتب results/experiments_real.json و results/learning_curve.png.

3. النتائج التي حصلنا عليها (بيانات KAIST الحقيقية، تسجيلات مُحْتَزَة)

العنوان الرئيسي (سليم مقابل بين اللغات، تدريب متوازن، اختبار طبيعي):

القناة	Macro-F1	المتوازنة الدقة
الاهتزاز	1.00	1.00
(تيار+اهتزاز) الاندماج	0.76	0.88
التيار	0.49	0.69

نتائج الحذف: - الموازنة ترفع استدعاء السليم من 0~ -> 1.00 على التيار (تمنع الانهيار)؛ لا أثر على الاهتزاز (مثالي أصلاً). - حجم الصورة يساعد التيار ترتيباً (0.68 -> 0.76 من 96 إلى 224؛ px الاهتزاز مُشَبَّع عند كل حجم (ف 96 px أسرع ~5 أضعاف، مجاناً). - منحني التعلم - الاهتزاز يبلغ 0.97 بنحو 46 صورة؛ التيار مسطح ~0.70 مهما زاد حجم البيانات -> سقفه جودة الإشارة لا كميته.

قيد أمين واحد: لا يوجد سوى 4 تسجيلات سليمة مميزة، فلا يمكن للنتيجة المثالية للاهتزاز أن تستبعد كلياً تعلم النموذج سمات خاصة بالتسجيل. هذا مذكور في كل مكان (التقرير، الشرائح، README).

4. الجودة وقابلية إعادة الإنتاج والتغليف

- 38 اختبار وحدة (python/tests/) تغطّي الضبط، المانيست، التقسيم (مع ضمان الفئات الصغيرة)، التقطيع، تحليل أسماء، الموازنة، أوزان الفئات، شكل المخطط الطيفي، المولد الاصطناعي، فهرسة محمل البيانات، النموذجين، إقران الاندماج، اختبار دخان للتدريب، ومقاييس التقييم. الاختبارات المحتاجة ل TF تُنَحْطَى نظيفاً عند غيابه.
- تكامل مستمر (github/workflows/ci.yml): يشغل الاختبارات على Python 3.10/3.11/3.12 عند كل دفع. شارة في README.
- أهداف Makefile: train-fusion, evaluate, train, split, scalograms, simulate, demo, test, setup, clean docs-ar, docs, report,
- CITATION.cff، رخصة MIT، و README.md شامل.
- GPU: فُعلت بطاقة NVIDIA Quadro P620 المحلية (tensorflow[and-cuda] install pip)؛ التدريب أسرع ~17 ضعفاً لكل خطوة من المعالج المركزي. النتائج متطابقة CPU مقابل GPU (السرعة فقط).

5. المخرجات المنتجة (docs/build/)

العربية	الإنجليزية	النوع
report-ar.docx ,report-ar.pdf	report.pdf , (29 ص)	التقرير
slides-ar.pdf ,slides-ar.pptx	slides.pdf ,slides.pptx	الشرائح

مصادر Markdown في docs/report/ و docs/presentation/؛ أعد البناء بـ docs make (إنجليزي) و docs-ar make (عربي)، RTL خط (Amiri).

6. دروس البيئة (لتمكّن من إعادة التشغيل)

- استخدم بيئة **Python 3.10 الافتراضية** (venv)؛ فـ python3 الافتراضي للجهاز هو 3.14 الذي لا يملك عجلة -Ten sorFlow.
- حاسوب التطوير يدخل السُّبَات عند الحمل ويعمل على البطارية، ما قتل المهام الخلفية الطويلة مرتين. الحل: شغل المهام الطويلة تحت systemd-inhibit -sleep-what ... وأبقه موصولاً بالكهرباء. كما يحفظ experiments.py نقاط تفتيش ويستأنف فلا يضيع شيء.
- يتعلق pip عند عجلة nvidia_cublas_cu12 الكبيرة - اجلب تلك العجلة وحدها بـ curl -L -C - ثم install pip، بعدها يكمل pip البقية.

7. الترتيب الزمني الدقيق لما بُني

1. تحليل المستودع + CLAUDE.md؛ مواصفات التصميم + خطة التنفيذ.
2. الهيكل البرمجي الأساسي (config)، manifest (split مع الاختبارات - TDD).
3. وحدة المخطط الطيفي (PyWavelets) + المولّد الاصطناعي -> demo make يعمل.
4. تدقيق البيانات -> اختيار، KAIST رفض Zenodo.
5. تمّحّل TDMS لـ KAIST المستخدم نزل المجموعة؛ استوعب 3,150 مقطعاً.
6. رسم المخططات الطيفية؛ إصلاح التقسيم لحالة الـ 4 تسجيلات السليمة.
7. تدريب التيار + الاهتزاز؛ اكتشاف انهيار الأكثرية؛ إضافة الموازنة + إصلاح خطأ التقييم.
8. بناء + تدريب نموذج الاندماج (إضافة GAP لإصلاح نفاذ الذاكرة).
9. تجارب الحذف (معزولة بعمليات فرعية، بنقاط تفتيش) -> experiments.md.
10. كتابة التقرير الكامل + الشرائح؛ بناء PDF/DOCX/PPTX.
11. إضافة CI، CITATION.cff؛ تفعيل GPU.

12. إنتاج التقرير + الشرائح بالعربية وصفحات غلاف ثنائية اللغة.

كل شيء مُودَع ومدفوع إلى github.com/molhamfetnah/pmsm-fault-diagnosis-cnn-scalogram